

Based on the requirements of your assignment brief and the search results available, here are 100 practice questions with answers, organized by the core technical areas of your project.

🧠 Agentic Design Patterns & Agent-to-Agent Communication

These questions explore how you implemented your Router, Orchestrator, Reflection, and Tool-Use patterns, as well as how your agents communicate .

1. What is an "agentic AI design pattern," and which ones did you implement?

> Answer: An agentic design pattern is a reusable architectural solution for structuring AI agents to solve complex, multi-step tasks autonomously . In my project, I implemented four patterns :

> 1. Router: Classifies the user's intent to route to the right workflow.

> 2. Orchestrator-Worker: Decomposes tasks and coordinates subordinate agents (e.g., Research Agent) .

> 3. Tool-Use: The Research Agent utilizes the RAG pipeline as a tool for information retrieval.

> 4. Reflection/Self-Critique: The Reflection Agent validates the final answer against the retrieved context to prevent hallucination.

2. Why is the Reflection pattern important for your Mental Health Assistant?

> Answer: In a mental health context, accuracy and safety are critical. The Reflection pattern acts as a safety net. It prevents the AI from hallucinating or confirming harmful delusions (sycophancy) by verifying that the generated response is strictly supported by the RAG-validated documents . This aligns with "validated RAG pipelines" used in enterprise settings to ensure reliability .

3. How do your agents communicate with each other?

> Answer: They use structured JSON messages passed through a central state machine (using LangGraph-inspired logic) . For example:

> * Router to Orchestrator: ``{ "intent": "literature_search", "query": "..."}``

> * Orchestrator to Research Agent: ``{ "task": "rag_search", "params": {"k": 5}}``

> * Research Agent to Reflection Agent: ``{ "status": "success", "retrieved_chunks": [...]}``

4. Explain the "Router" pattern and the model you used for it.

> Answer: The Router is an agent that classifies the user's input into specific intents (e.g., `literature\_search`, `gap\_analysis`). For this, I deliberately chose Groq's `Llama 3.1 8B` because routing requires low latency and is cheap to run, making it ideal for this simple, high-volume task.

5. What is the "Orchestrator-Worker" pattern?

> Answer: The Orchestrator decomposes a complex user request into a plan of smaller sub-tasks (Task Decomposition) and delegates these to specialized "Worker" agents (e.g., Research Agent). This follows the pattern of "planner vs executor vs critic agents".

6. What is the "Tool-Use" pattern and what tools did you use?

> Answer: The Tool-Use pattern allows an agent to call external functions or APIs to gather information. My `ResearchAgent` uses the `RAG pipeline` and the `ChromaDB vector store` as its primary tools to retrieve relevant document chunks.

7. What is "sycophancy" in AI agents and how did your project prevent it?

> Answer: Sycophancy is when an AI model confirms everything a user says to maximize user satisfaction, regardless of factual or psychological correctness. My Reflection pattern prevents this by checking if the answer is grounded in the provided RAG context. If not, it flags the response as insufficient, avoiding the generation of harmful content or the reinforcement of delusions.

8. Why is an "empathetic" AI dangerous for users in a mental health crisis?

> Answer: Simulated empathy can create psychological dependence and blur the line between AI and human connection. An AI that is always available and confirms everything a user says can prevent a vulnerable person from seeking help from real professionals or can even reinforce psychotic delusions. This is why Reflection and safety guardrails are non-negotiable.

9. How did you design for human oversight and error recovery?

> Answer: My system handles errors gracefully in the UI. If an agent fails (e.g., a 401 API error), a user-friendly error message is displayed, rather than the system crashing. This provides a built-in "human-in-the-loop" signal for the user to act upon.

10. What happens when the Reflection Agent "flags" a response?

> Answer: If the Reflection Agent determines the RAG-retrieved context lacks sufficient information to answer the query accurately, it flags the response and returns a "Warning"

message to the user (e.g., " ⚠ Based on the available documents, I cannot find sufficient information..."). This protects the user from a hallucinated answer .

🔍 RAG & Model Selection Strategy

These questions target your deliberate choice of models (Groq vs. OpenRouter) and the design of your RAG pipeline.

11. Why did you use two different models (Groq and OpenRouter) instead of just one?

> Answer: This is a cost-latency-quality trade-off. Using one large model for everything is expensive and slow. For simple routing, I use Groq's `Llama 3.1 8B` which is fast and free. For the complex task of synthesis and validation, I use a more powerful model (like `Gemini 2.0 Flash` or `Llama 3.1 70B` via OpenRouter) where the higher quality justifies the cost .

12. How did you justify choosing `Llama 3.1 8B` (Groq) for your Router?

> Answer: Intent routing is a low-latency, low-reasoning task. `Llama 3.1 8B` at Groq provides very low latency (< 1s), is near-free, and has a sufficient context window for simple prompts. It's an efficient choice for a gatekeeper function .

13. What is RAG (Retrieval-Augmented Generation) and why did you need it?

> Answer: RAG grounds the LLM's response in a specific, domain-specific corpus (my 20+ PDFs). Without RAG, the AI would only have its general world knowledge and would "hallucinate" answers. RAG ensures the agent's answers are based on the factual data provided (e.g., clinical guidelines, research papers) .

14. What was your chunking strategy and why?

> Answer: I used a `RecursiveCharacterTextSplitter` with a chunk size of 1000 characters and an overlap of 200 characters. This size is standard for embedding models and ensures each chunk contains enough semantic context. The overlap prevents important information from being lost when cutting between chunks.

15. What embedding model and vector store did you use?

> Answer: I used `all-MiniLM-L6-v2` from HuggingFace as it's lightweight, fast, and produces high-quality embeddings for semantic search. For the vector store, I used ChromaDB as it's a

free, open-source, persistent vector database that integrates seamlessly with LangChain and runs locally .

16. How would you evaluate the performance of your RAG pipeline?

> Answer: I ran 5 sample queries and evaluated the `relevance\_score` of the retrieved chunks. I checked if the chunks' semantic content (e.g., "What is PHQ-9?") actually matched the user's question. This retrieval evaluation is documented in my README .

17. What is the "deadly triad" in the context of AI learning and does it apply to RAG?

> Answer: The "deadly triad" is a concept in Reinforcement Learning involving Function Approximation, Bootstrapping, and Off-policy learning . It doesn't directly apply to standard RAG pipelines. However, the concept of instability is similar in RAG: if the retrieval provides bad context (noise) and the model hallucinates, this creates a dangerous feedback loop. This is why validation (Reflection) and "validated RAG pipelines" are crucial .

18. What are the benefits of using an LLM for depression risk prediction over a traditional ML model like Random Forest?

> Answer: LLMs (like those in my agents) have a larger context window and can interpret complex, nuanced language (text) in addition to behavioral data . While a Random Forest might be great for structured data (e.g., steps, sleep) , an LLM can understand the *context* of unstructured user input (like chat logs) to generate a more holistic risk assessment .

19. Why is recall (sensitivity) a more important metric than accuracy for your application?

> Answer: In mental health detection, missing a positive case (False Negative) is far more dangerous than a false alarm (False Positive) . A False Negative could mean a depressed user doesn't receive help. Therefore, maximizing recall (capturing all depressed users) is critical to ensure safety, even if it means some false positives .

20. What is a "validated RAG pipeline" and how is it different from a standard one?

> Answer: A validated RAG pipeline goes beyond just retrieving documents; it adds a layer of validation to ensure citations are correct and the answer is reliably grounded . My Reflection agent performs this validation, giving the response a high degree of auditability, which is crucial for a project in the mental health domain .

🧑‍💻 Deployment, GitHub & Technical Practices

21. How do you secure your API keys in a Streamlit app?

> Answer: I use Streamlit Secrets Management. The API keys are stored in a `secrets.toml` file, which is excluded from version control via `.gitignore`. This ensures the keys are never exposed in the source code or commit history, a mandatory requirement for the assignment.

22. What is the purpose of a `.gitignore` file?

> Answer: The `.gitignore` file tells Git which files and folders to ignore and not track in version control. This is critical for excluding virtual environments, cache files (`__pycache__`), and API secrets (`secrets.toml`) to keep the repository clean and secure.

23. Explain your Git branching strategy for this project.

> Answer: I used feature branches. For example, `feature/rag-pipeline`, `feature/agent-orchestration`, and `feature/streamlit-ui`. Each branch focuses on a single, logical piece of work. These were then merged into the `main` branch via Pull Requests (PRs) with descriptive titles, ensuring a clean and organized development history.

24. What kind of commit messages did you use and why?

> Answer: I used semantic commit messages (`feat:`, `fix:`, `docs:`, `refactor:`, `test:`). This practice is common in professional software engineering as it makes the commit history easier to read and understand, allowing anyone to quickly see the nature of the changes at a glance.

25. Why are 15+ incremental commits important for your assignment?

> Answer: It shows organic, iterative development over time rather than a single bulk upload of all files. A history with many small commits proves that the code was developed step-by-step, aligns with professional standards, and allows for easier code reviews and rollbacks.

26. How does your system handle a user who types "I don't want to live anymore."?

> Answer: While not a primary feature, the system acts as a research assistant. The Reflection agent will check if the RAG knowledge base contains any relevant crisis information. If it does, it can cite it. If not, the Reflection agent will flag the response, but also return a standard safety disclaimer, directing the user to professional help.

27. What is "AI-first" workflow design?

> Answer: In the context of my project, "AI-first" means workflows are reimagined with AI agents handling the heavy lifting (retrieval, synthesis, validation) while the human user remains the

"owner and steerer" of the end-to-end outcome by asking the initial question and evaluating the final answer .

28. Why did you use LangChain in your project?

> Answer: LangChain provides a robust, open-source framework for building applications powered by LLMs. It simplifies complex tasks like document loading, text splitting, and creating vector stores, allowing me to focus on the high-level agentic logic rather than low-level integrations .

29. What is the difference between High-Level Design (HLD) and Low-Level Design (LLD)?

> Answer: HLD is the system architecture (the WHY and WHAT), like defining your agent roles and workflow . LLD is about implementation details (the HOW), such as specific prompt contracts, retry logic, and token budgeting that make your system production-ready .

30. What would you do if your agent started looping (repeating the same chain of actions)?

> Answer: Agent looping often indicates a state machine that is not correctly evolving . I would implement a maximum iteration counter in the orchestrator's execution loop. If the counter is exceeded, the orchestration process would be forcibly halted, an error would be logged, and a fallback message would be shown to the user.

 General Agentic AI Knowledge

31. What is the difference between forward chaining and backward chaining?

> Answer: Forward chaining is data-driven (starts with facts, derives conclusions), while backward chaining is goal-driven (starts with a hypothesis, works backward to find supporting facts) . My Router uses a form of forward chaining to decide the intent.

32. What is the "conflict set" in an inference engine?

> Answer: The conflict set is the collection of all rules whose conditions are matched by the current facts. Conflict resolution is needed to decide which one to fire first. Recency (favoring newer rules) is a common strategy, where specificity (favoring more specific rules) could also be used .

33. What is the "exploration vs exploitation" trade-off?

> Answer: This is a core concept in Reinforcement Learning (RL) . It addresses the balance between trying new strategies (exploration) to find better outcomes and using known strategies (exploitation) to maximize reward. It applies to agentic systems when an agent is deciding which tools to use.

34. What is ReAct (Reasoning + Acting) and why is it a powerful pattern?

> Answer: ReAct is a prompting strategy that combines chain-of-thought reasoning with action-taking. The agent thinks about a problem, acts (e.g., queries a tool), and reflects on the result in a loop. It allows for dynamic problem-solving and is more robust than pure reasoning, making it ideal for tool-use agents.

35. What is an "Epsilon-Greedy" strategy?

> Answer: A simple but effective strategy to balance exploration and exploitation . With probability ϵ , the agent takes a random action (exploration). With probability $1 - \epsilon$, it takes the best-known action (exploitation). This method is very common in Deep RL algorithms like DQN .

36. Why is memory management in agents a product decision, not just a prompt trick?

> Answer: How you manage memory (short-term vs long-term) fundamentally affects the user experience . For a mental health assistant, remembering a user's previous concerns is a product feature, not just a prompt to add to the context window. It determines the depth of the conversation and the perceived intelligence of the agent .

37. What is the "agentic organization" paradigm?

> Answer: It's a new organizational model where humans and AI agents collaborate side-by-side at near-zero marginal cost . Companies that adopt this can achieve revenue per employee levels similar to AI-native startups, as workflows shift from centralization to flat, agentic networks .

38. What is the "gen AI paradox"?

> Answer: It's the observation that while nearly 80% of companies are using generative AI in some form, they are seeing zero bottom-line impact . This highlights the "pilot-to-production" gap. My project aims to bridge this gap through a functional, testable agentic system.

39. What is the difference between a "tool" and an "agent"?

> Answer: An agent is an autonomous entity capable of reasoning and making decisions (e.g., using an LLM) . A tool is a specific function that an agent can call to perform a task (e.g., a RAG query, a web search, an API call). The agent decides when and how to use the tools .

40. Why is judging an AI engineer's ability to "debug systems" becoming more important than their knowledge of models?

> Answer: Because models are becoming commoditized, the real value and difficulty lie in building reliable, safe, and scalable systems . Being able to diagnose why a RAG pipeline is returning bad context or why an agent is looping is a much rarer and more valuable skill than knowing the specs of a model.

💬 Project-Specific Conceptual Questions

41. How does your system prevent "sycophancy" (AI confirming user delusions)?

> Answer: Sycophancy is prevented by the Reflection Agent, which compares the LLM's proposed answer against the actual RAG context. If the answer goes beyond or contradicts the context, it is flagged and rejected, preventing the AI from uncritically confirming harmful statements .

42. What are the limitations of using a "small" LLM for depression prediction?

> Answer: Small LLMs (e.g., 7B-8B parameters) can struggle with complex linguistic nuance and may have lower reasoning capacity than larger models, potentially leading to a higher rate of false negatives . However, their advantage lies in lower latency and the ability to run on-device, which is better for privacy .

43. How do you ensure "Domain Filtering" in your RAG pipeline?

> Answer: Domain filtering is ensured by the initial data selection (the 20+ PDFs are all domain-specific) and the retrieval mechanism itself. The embeddings are designed to only search within that specific corpus, ensuring the agent's knowledge is bounded to the provided documents and doesn't pull information from the open web.

44. What is "Validation" in the context of a RAG pipeline?

> Answer: Validation ensures the generated answer is not only grammatically correct but is also factually supported by the retrieved documents . My Reflection Agent performs this by checking if the synthesized answer can be directly traced back to the RAG chunks, providing a form of citation auditability .

45. Why is "context window" an important consideration for your Reflection Agent?

> Answer: The Reflection Agent needs a large enough context window to analyze the user's query and the top 3-5 retrieved chunks simultaneously to make a judgment. If the model has a small context window, it won't be able to "see" all the evidence, making its validation less reliable .

46. How does data from sensors (behavioral) differ from data from text (symptoms)?

> Answer: Behavioral data (sleep, steps, phone usage) is objective but lacking in context . Textual data (symptom description) is rich in context but subjective . An ideal system, like mine, uses RAG to provide the textual context, but the current iteration focuses on the RAG pipeline. My future work aims to integrate sensor data for more accurate multimodal depression risk prediction .

47. What is the "duty of care" concept in AI liability?

> Answer: Providers of AI systems for emotional support have a "duty of care" to ensure their product does not cause psychological harm . Under regulations like the EU AI Act, a high-risk AI system would require risk management, human oversight, and incident reporting. Disclaimers alone are not sufficient to absolve liability if the system contributes to a user's harm .

48. What is the "pilot-to-production" gap in AI?

> Answer: This refers to the phenomenon where 90% of AI use cases never progress from a proof-of-concept or pilot phase to being fully deployed and used in production . My project demonstrates an end-to-end deployment to Streamlit Cloud, showcasing the ability to take an idea from prototype to a live, publicly accessible system.

49. Why are "validated RAG Pipelines" a growing trend in enterprise AI?

> Answer: Enterprises need to trust the output of AI systems before they can use them in critical operations . Validated pipelines provide a layer of auditability and trust by ensuring answers are verifiably grounded in company data, reducing the risk of cost or reputation damage from hallucinated output.

50. What is the "Elastic Digital Worker" concept?

> Answer: An elastic digital worker is a specialized AI agent that can be deployed instantly to handle tasks that would normally require hiring a human worker . My agents (Router, Orchestrator, Research) act as such workers, operating 24/7 to provide research assistance on depression risk prediction.

📁 Core Technical & Coding Questions

These questions assess your understanding of the specific code you wrote.

51. What is a Streamlit session state and why didn't you use it?

> Answer: `st.session_state` is used to share data between reruns. My app is stateless; each query runs a new workflow, so it wasn't strictly necessary. However, future work could use it to store conversation history for the agent's memory.

52. How would you implement a "history" or memory feature in your application?

> Answer: I would store past interactions in a database (e.g., SQLite). I would then write a new agent that retrieves a summary of past interactions (using RAG on the history table) and injects it as a "System" prompt for the Orchestrator or Reflection Agent, giving it context for the new question .

53. How do you handle rate limiting from API providers?

> Answer: I use `try-except` blocks to catch API errors. A more robust implementation would use a library like `tenacity` to implement exponential backoff, automatically retrying the request after a short delay when a rate limit error occurs.

54. What is a "Yanked" package and why did you get an error?

> Answer: A "yanked" package is a version of a Python package that has been removed from the PyPi repository . I got the error because the specific version number for `langchain-community` I specified in the initial `requirements.txt` was yanked. I fixed it by allowing a later version `>=0.1.0` .

55. What are some strategies to reduce the latency of an agentic workflow?

> Answer: Strategies include: (1) Using faster models for simpler tasks (Router), (2) Caching results of vector DB queries, (3) Reducing the number of chunks retrieved, and (4) Using Groq for its fast inference speed .

56. How would you implement "tool governance" to ensure safe tool execution?

> Answer: I would enforce the following: (1) Scoped Permissions: Agents only have access to the tools they need (e.g., Research Agent only has RAG access). (2) Audit Trails: Logging all tool calls and API interactions. (3) Human-in-the-loop: Requiring user confirmation for certain actions .

57. What is the benefit of using a "developer message" over a "system message" in the OpenAI API?

> Answer: Developer messages are prioritized over user messages, ensuring the core instructions (e.g., "You are a strict validator... Return JSON") are not overridden by user inputs, improving prompt adherence .

58. What is the purpose of a "target network" in DQN?

> Answer: A target network is a copy of the main Q-network updated periodically to provide a stable target for Q-learning . It prevents the "moving target" problem that causes training to destabilize, which relates to the concept of how I use a "Reflection Agent" to provide a stable target for validation .

59. What is a "circular buffer" and how did you use it?

> Answer: A circular buffer is a fixed-size data structure where new data overwrites the oldest data . It's used in the coding assignment for a Replay Buffer to store experiences, ensuring the agent's memory stays within a fixed size. I used a similar logic in my ChromaDB where I limited the number of retrieved chunks.

60. Why is the final layer of your Q-Network lacking an activation function?

> Answer: Q-values are scalar estimates of cumulative return, which can be positive or negative unbounded values . Applying a squashing activation (like sigmoid or tanh) would artificially restrict them and introduce bias. The raw linear output is more appropriate for a regression-like loss function.

🧪 Testing & Troubleshooting

61. What is the significance of "retrieval relevance scores" (e.g., 0.85) in your RAG output?

> Answer: The retrieval relevance score is a cosine similarity score, measuring the semantic distance between the user's query and a given chunk. A score closer to 1.0 (e.g., 0.85) means the chunk is highly relevant. This score is used to filter out irrelevant chunks and is a key metric for evaluating RAG performance.

62. Why did you get a "No module named 'groq'" error and how did you fix it?

> Answer: This error occurred because the `groq` Python package was not installed in my active virtual environment. I fixed it by activating my `venv` and running `pip install groq` to install it locally.

63. Why did your app first run fine but then fail with an "API key" error?

> Answer: The API key error usually occurs if the API key is invalid (e.g., regenerated) or if the `secrets.toml` file is not in the correct place. I had to regenerate the key to resolve the invalid credential.

64. Why is evaluating "recall" more important than "accuracy" for your system?

> Answer: In mental health detection, missing a positive case (False Negative) is far more dangerous than a false alarm (False Positive). A False Negative means a depressed user doesn't receive help. Therefore, maximizing recall (capturing all depressed users) is critical to ensure safety, even if it means some false positives .

65. How do you choose between `Chunk\_size=500` vs `1000`?

> Answer: A chunk size of 500 tokens is better for precision but can lose context, while 1000 tokens is better for context but can pick up noise. I chose 1000 because it provides enough surrounding text for the LLM to understand the full context while still being concise enough for efficient semantic search .

66. What is the difference between semantic and lexical search?

> Answer: Lexical search (e.g., BM25) matches keywords, while semantic search (Vector Search) matches concepts and meaning. My system uses semantic search (via embeddings) to find chunks that are conceptually similar to the user's query, even if they don't share the same exact words.

67. What is the "deadly triad" of RL and how might it apply to your system?

> Answer: The "deadly triad" (Function Approximation, Bootstrapping, Off-policy) can cause RL training to destabilize . Although my system is not an RL system, the concept of instability applies: if RAG provides bad context and the model hallucinates to get a "good" user rating, it's a dangerous feedback loop, which is why validation (Reflection) is critical.

68. What is the "exploration vs exploitation" trade-off?

> Answer: This is a core concept in Reinforcement Learning . It represents the balance between trying new strategies (exploration) to find better outcomes and using known strategies (exploitation) to maximize reward. For example, my Router Agent could "explore" by occasionally misclassifying a query to learn to handle new intents.

69. What is the "pilot-to-production" gap?

> Answer: This term describes the phenomenon where a majority (up to 90%) of AI use cases never progress from a pilot or proof-of-concept phase to a fully deployed, production-ready system . My project aimed to bridge this gap by deploying a working application to Streamlit Cloud.

70. What is the "Agentic AI Mesh" architecture?

> Answer: The "Agentic AI Mesh" is an architecture paradigm for governing a rapidly evolving organizational AI landscape . It suggests that AI agents should operate as a mesh of interconnected, decentralized services rather than a centralized hub, which resembles how my agents (Router, Orchestrator, Research) act as an interconnected network.

🛠️ Advanced & Future Work Concepts

71. What would you do to make your system "production-ready"?

> Answer: To make the system production-ready, I would add robust monitoring (logging API calls and errors), implement rate limiting and retry logic, and set up a feedback loop to collect user ratings on answers to continuously improve performance . I would also add a "human-in-the-loop" feature for high-risk outputs .

72. How could this system be extended to predict changes in depression levels?

> Answer: I could implement a prediction pipeline that uses a time-series model (e.g., an LSTM) or a standard ML model on behavioral features . The new agent would analyze this data and be prompted with "few-shot" examples to predict if the user's PHQ-4 score will increase or not .

73. What are the ethical considerations of using AI for depression risk prediction?

> Answer: Key ethical considerations include Privacy: ensuring user data is secure and not misused; Bias: ensuring the model is fair across different demographics; Transparency: users should know they are talking to an AI; and Safety: ensuring the system does not provide harmful advice or create psychological dependence .

74. What is the "gen AI paradox" and why are 80% of companies not seeing value?

> Answer: The paradox is that while 80% of companies are using Generative AI, they see no bottom-line impact . This is because many are using it for small, isolated tasks rather than reinventing end-to-end workflows . My project demonstrates end-to-end reinvention by automating the entire research and synthesis workflow.

75. What is the "OODA Loop" and how does it relate to agentic AI?

> Answer: The OODA Loop (Observe, Orient, Decide, Act) is a military strategy framework. In Agentic AI, the "Reflection" pattern is analogous to the "Observe and Orient" phases, where the agent observes its actions and the environment, reorients its strategy, decides on a new action, and acts again until a goal is achieved.

76. What is a "LLM Router" and why is it more efficient than one model?

> Answer: An LLM Router is a mechanism that sends different tasks to different models. It is more efficient because small, fast models (like Groq's Llama 3.1 8B) can handle simple tasks cheaply, while larger, more capable models (like GPT-4) are reserved for complex reasoning tasks, optimizing for cost and latency .

77. What is "Task Decomposition" and how did you implement it?

> Answer: Task Decomposition is a planning strategy where a complex user request is broken down into smaller, manageable sub-tasks . I implemented this in the Orchestrator agent, which creates a "Plan" (e.g., Step 1: Research, Step 2: Validate) and coordinates workers to complete them.

78. What is "Token Budgeting" and why is it important for your app?

> Answer: Token budgeting is the practice of managing the number of tokens you send to an API to control costs and latency . For example, limiting the number of RAG chunks sent to the Reflection Agent prevents the prompt from becoming too long and expensive. It is a critical low-level design consideration for production systems .

79. How would you build a real-time agentic system for monitoring mental health?

> Answer: I would create an "Observer" agent that continuously ingests data streams (like passive sensor data or typing patterns). This agent would trigger a "Crisis" agent if it detects anomalies in the data . The Orchestrator would then manage the sequence of events and potentially alert a human supervisor .

80. What are the benefits of using a "small" LLM for depression prediction?

> Answer: Small LLMs are advantageous for their speed and the ability to run on-device, ensuring privacy . They can be deployed directly on a user's phone without sending sensitive

data to external servers, which is crucial for mental health applications where privacy is of utmost concern .

📄 More Project-Specific Questions

81. How do you handle a user who simply asks "What is PHQ-9?" vs "How does PHQ-9 compare to GAD-7?"?

> Answer: The Router agent classifies these differently. The first is classified as ``literature_search`` (definition), while the second is classified as ``gap_analysis`` (comparison). The Orchestrator creates a different task plan based on this intent, showing the adaptive nature of the workflow.

82. Why does your RAG pipeline use 1913 chunks?

> Answer: This number is a result of the document chunking process. The system reads the 20+ PDFs, which contain 370 pages of text, and splits them into chunks of 1000 characters with 200-character overlap. The final count of 1913 chunks provides a rich, granular, and searchable knowledge base.

83. What is the trade-off between using ``Chunk_size=500`` vs ``1000``?

> Answer: A chunk size of 500 tokens is better for precision (fewer irrelevant chunks) but can lose context. A size of 1000 tokens preserves more context but can pick up more noise . I chose 1000 because for clinical documents, context is crucial for understanding the data.

84. What would happen if the user asks a question about "CKD" (Chronic Kidney Disease) but you have no data?

> Answer: The RAG retrieval would search for CKD and find no relevant chunks. The Reflection Agent would then check the results and flag the response as "insufficient data," returning a message like: "Based on the available documents, I cannot find sufficient information to answer this query."

85. What are the legal implications if your system provides harmful medical advice?

> Answer: Under the EU AI Act, if my system is classified as "high-risk," it would require risk management, human oversight, and incident reporting . To mitigate this, I include a disclaimer and use the Reflection Agent to ensure answers are strictly grounded in the provided RAG data.

86. How do you evaluate the "quality" of your RAG retrieval for a specific query?

> Answer: I evaluate quality by checking the relevance scores of the top retrieved chunks. A high score (e.g., >0.8) for the top 3 chunks indicates a successful retrieval. I also conduct manual qualitative analysis: does the retrieved text semantically match the query?

87. How would you add "memory" to your agent?

> Answer: I would implement a short-term memory by storing the current conversation history in the Streamlit session state and a long-term memory by persisting interactions to a vector database (like ChromaDB). A "Memory Agent" would then retrieve relevant past interactions to provide context to the current query.

88. What is "sycophancy" and how did your project prevent it?

> Answer: Sycophancy is when an AI model confirms everything the